

Lambda詳細設計書（レンタルスタジオサーチアプリ）

本設計書は、レンタルスタジオサーチアプリにおけるAWS Lambda関数単位の責務・入出力・処理フローを定義する。対象は4ブランド（BUZZ／studio worcle／サウンドスタジオNOAH／スタジオミッション）の実店舗に限定し、AIによるスコアリング機能（おすすめTOP3・スコア計算・generate_studio_score.py・studio-recommendationsテーブル）は完全に廃止した後の構成を反映している。旧構成・変更の経緯は[企画・設計アーカイブ.html](#)を参照。

1. 設計方針

- 1 Lambda = 1責務（単一責任原則）
- API系とバッチ系を分離（SAMの別CodeUriパッケージ、コード共有不可）
- DynamoDBアクセスは原則Lambda経由（ただし空き状況の書き込みのみ例外。後述）
- リアルタイム処理は禁止（バッチ・ローカルスクリプト中心）
- 実在性に関わる情報（座標・スタジオ名・部屋の広さ・料金）はGoogle Places APIまたは公式サイトの実テキストのみを情報源とし、AIには生成させない
- AI（Claude）はAI相談チャットの応答生成にのみ用いる。スコア計算・おすすめ順位付けは一切行わない（シンプルな条件絞り込みのみ）

2. Lambda一覧（API系）

関数名	役割	トリガー	認証
getStudiosHandler	4ブランドのスタジオ一覧取得	API Gateway（GET /studios）	不要

getStudioAvailabilityHandler	指定日の 空き状況 取得	API Gateway (GET /studios/{studioId}/availability)	不 要
postAnalyticsEventHandler	詳細表 示・予約 クリック の計測イ ベント記 録	API Gateway (POST /analytics/event)	不 要
getEventsHandler	「次のイ ベント」 一覧取得	API Gateway (GET /events)	必 須
postEventsHandler	「次のイ ベント」 新規登録	API Gateway (POST /events)	必 須
deleteEventHandler	「次のイ ベント」 削除	API Gateway (DELETE /events/{eventId})	必 須
getRecommendedStudiosHandler	イベント 条件（広 さ・空き 状況）に 合う部屋 の絞り込 み	API Gateway (GET /recommend-studios)	必 須
putStudioImageHandler	スタジオ 写真URL の設定	API Gateway (PUT /studios/{studioId}/image)	必 須
getPostsHandler	レビュー 取得	API Gateway (GET /posts)	不 要
postPostsHandler	レビュー 投稿の作	API Gateway (POST /posts)	必 須

	成（★評価含む）		
putPostHandler	レビュー 投稿の編集（本文・★評価・写真）	API Gateway（PUT /posts/{postId}）	必須
deletePostHandler	レビュー 投稿の削除	API Gateway（DELETE /posts/{postId}）	必須
postFavoritesHandler	お気に入り登録	API Gateway（POST /favorites）	必須
deleteFavoritesHandler	お気に入り削除	API Gateway（DELETE /favorites/{studioId}）	必須
getFavoritesHandler	お気に入り取得	API Gateway（GET /favorites）	必須
postPresignUploadHandler	S3アップロード用の署名付きURL発行	API Gateway（POST /uploads/presign）	必須
postChatHandler	AIチャットへのメッセージ送信・Claude応答生成・メッセージ編集	API Gateway（POST /chat）	必須
getChatsHandler	チャット履歴一覧取得	API Gateway（GET /chats）	必須

getChatHandler	特定チャットの全メッセージ取得	API Gateway (GET /chats/{chatId})	必須
deleteChatHandler	チャットの削除	API Gateway (DELETE /chats/{chatId})	必須
getMyProfileHandler	自分の表示名（プロフィール）取得	API Gateway (GET /me)	必須
putMyProfileHandler	表示名の設定・更新	API Gateway (PUT /me)	必須

3. Lambda一覧（バッチ系）

関数名	役割	トリガー
discoverStudiosBatch	Google Places APIで4ブランドの新規店舗候補を探索しStudiosテーブルへ追加	adminRunStudioDiscovery 経由のみ（自動スケジュールなし）
adminRunStudioDiscovery	フロントの「現在地から探す」ボタンから discoverStudiosBatch を起動する薄いラッパー。1日10回のレート制限	API Gateway (POST /admin/run-studio-discovery、認証必須)
sendAnalyticsDigestBatch	直近24時間の閲覧数・予約クリック数を集計しメールで通知	EventBridge（毎日AM6:30 JST）

generateStudioScoreBatch と adminRunAiBatch は完全に削除済み。 スコア計算・AI推薦理由生成・毎日AM6:00 JSTのEventBridgeルール・ POST /admin/run-ai-

batch エンドポイントは、AIスコアリング機能の廃止にともないすべて削除した。以後「AI分析を実行」という操作自体がアプリから無くなっている。

4. 個別設計

getStudiosHandler

目的：4ブランド（BUZZ／worcle／NOAH／スタジオミッション）のスタジオ一覧を返却する。StudiosTableを全件scanし、brand属性が対象4ブランドに含まれるものだけにフィルタする（_is_supported_brand()）。ヨガ・ピラティス系（4ブランド導入以前に Google Places発見で混入していた候補）も同時に除外する。

出力（1件分の例）

```
{
  "studioId": "buzz-kichijoji",
  "brand": "buzz",
  "name": "BUZZ 吉祥寺スタジオ",
  "address": "...",
  "lat": 35.7, "lng": 139.5,
  "sourceUrl": "https://www.buzz-st.com/...",
  "rooms": [
    {
      "roomName": "Aスタジオ",
      "areaSqm": 45.0,
      "secondDimensionLabel": "鏡幅",
      "secondDimensionM": 6.0,
      "minPriceYen": 3000,
      "reserveUrl": "https://www.buzz-st.com/reserve/...",
      "photoUrls": ["https://.../a1.jpg", "https://.../a2.jpg"],
      "floorPlanUrl": "https://.../a-floorplan.png",
      "equipment": ["フローリング", "音響設備", "鏡張り"]
    }
  ]
}
```

getStudioAvailabilityHandler

目的：指定日のスタジオの空き状況を返却する。dateクエリパラメータ（YYYY-MM-DD）は必須で、無ければ400を返す。AvailabilityTable（PK: studioId, SK:

date) を `get_item` し、該当データが無い場合は**404にはせず** `rooms: [], scrapedAt: null` で200を返す（「まだスクレイピングされていないだけ」を明示するための設計判断）。

処理フロー

1. `pathParameters.studioId`, `queryStringParameters.date` を取得（dateが無ければ400）
2. `AvailabilityTable.get_item(Key={studioId, date})`
3. Itemがあれば`rooms`と`scrapedAt`を、無ければ `rooms: [] / scrapedAt: null` を返す

getEventsHandler / postEventsHandler / deleteEventHandler

目的：ユーザーが登録する「次のイベント」（発表会・公演などの本番情報）の CRUD。EventsTable（PK: `userId`, SK: `eventId`）に対する操作で、いずれもCognito 認証が必須。

登録項目（POST /events）

`title` (str, 必須) : イベント名
`stageWidthM` (number, 必須): 本番ステージの横幅（メートル）
`stageDepthM` (number, 必須): 本番ステージの奥行き（メートル）
`performerCount` (int, 必須) : 出演人数
→ `createdAt/updatedAt` を付与してEventsTableへ`put_item`

GET /events は`userId`で`query`し、`updatedAt`降順でソートして返す。DELETE /events/{eventId} は`Key={userId, eventId}`で`delete_item`（所有者以外の`eventId`を指定しても`userId`が一致しないため削除は成立しない設計）。

getRecommendedStudiosHandler

目的：登録済みイベントの本番実寸・出演人数と、目的（振り入れ／構成）・希望日時から、【広さ条件を満たす】かつ【指定時間帯に空きあり】の部屋のみを返す。スコアリングは行わない、単純な絞り込みロジックである点が設計上の核。

必要な広さの計算（目的による余裕度の違い）

`REQUIRED_SQM_PER_PERSON = {"振り入れ": 2.5, "構成": 1.8}` # 1人あたりの必要面積 (m2)

```
必要面積 = performer_count * REQUIRED_SQM_PER_PERSON[purpose]
```

振り入れ（新しい振付を覚える段階）は鏡の前で細かい確認をするため、出演人数に対して 部屋がゆったりしているほど良く1人あたり2.5㎡を要求する。構成（立ち位置・隊形確認の 段階）は隊形移動のスペースが取れば足りるため、振り入れほどの余裕は求めず1.8㎡としている。

処理フロー

1. クエリパラメータ (eventId, purpose, date, startTime, durationMinutes省略時120分) を取り
・ 必須パラメータ欠如は400、purposeがEVENT_PURPOSES外なら400
2. EventsTableからeventIdに該当するイベントを取得 (Key={userId, eventId})。無ければ404
3. StudiosTableを全件scanし、対象4ブランド・除外施設名でフィルタ
4. 各スタジオの各roomについて:
 - a. _room_meets_size(room, performer_count, purpose)
→ room.areaSqmが無ければ判定不能としてFalse。あれば必要面積と比較
 - b. _room_has_availability(...)
→ AvailabilityTableをget_item。データ無し／部屋データ無し／
指定時間帯を覆うスロットが1件も無い場合はFalse
(「わからない」を「空いていない」として安全側に倒す)
→ start_timeからdurationMinutes分の間の全スロットがavailable=trueならTrue
 - c. 両方満たす {studio, room} の組だけmatchesに追加
5. {"items": matches} を返却

discoverStudiosBatch (discover_studios.py)

目的：Google Places APIで4ブランドの新規店舗候補を探索し、Studiosテーブルに追加する。adminRunStudioDiscovery（「現在地から探す」ボタン経由、位置指定あり）から呼ばれる。

検索クエリと店名フィルタ

```
TARGET_BRAND_QUERIES = [  
    "BUZZ レンタルスタジオ 東京", "studio worcle", "サウンドスタジオNOAH", "スタジオミッシ  
]  
NATIONWIDE_QUERIES = TARGET_BRAND_QUERIES  
NEARBY_QUERY = TARGET_BRAND_QUERIES # 位置指定あり検索でも同じ4ブランド限定クエリ集合を
```

detect_brand(name) が、Text Searchで見つかった候補の店名を BRAND_NAME_KEYWORDS
(brand→店名キーワードのdict) と照合し、4ブランドのいずれにも一致しない候

補は登録しない。クエリ自体を4ブランド名に 限定していても、類似名の無関係な店舗が結果に混ざりうるため、店名レベルでの 二重チェックとして機能している。

処理

Studios全件取得（重複判定用）

- Google Places API Text Search（4ブランド限定クエリ、位置指定時はlocation/radius(15km)
- detect_brand() で店名が4ブランドいずれかに一致するものだけを候補として残す
- 既存スタジオと緯度経度がhaversine距離300m以内なら重複として除外
- 新規候補ごとにStudiosテーブルへput_item（studioIdは”{brand}-{shop}”形式、placeIdも保

座標・スタジオ名などの実在性に関わる情報はGoogle Places APIの結果のみを採用し、LLM（Claude）には生成させない（存在しないスタジオをAIが作り出すハルシネーションを避けるため）。以前存在していた「Claudeによる設備・収容人数の推測」「公式サイトからの料金抽出」は、スコアリング機能の廃止にともない削除した。部屋の広さ・料金・設備等は backend/scripts/scrape_availability.pyが公式サイトから直接取得する。

backend/scripts/scrape_availability.py（ローカル専用CLIスクリプト、Lambdaではない）

目的：4ブランドの公式サイトから、空き状況・部屋の広さ・部屋写真・平面図・設備をスクレイピングし、StudiosTable（部屋マスタ）とAvailabilityTable（日次の空き状況）へ 直接書き込む。クラウドにはデプロイせず、手動またはローカルのタスクスケジューラで 定期実行する運用としている。

ブランドごとの取得方式の違い

ブランド	取得方式	備考
BUZZ	BeautifulSoup（静的HTML）	4ブランド中唯一、平面図画像（floorPlanUrl）を取得できる
studio worcle	Playwright（JS描画）	空き状況カレンダーがJavaScriptで描画されるため、ブラウザ自動操作が必要

サウンド スタジオ NOAH	BeautifulSoup (一部静的 HTML)	ページによりPlaywrightを併用
スタジオ ミッショ ン	Playwright (JS描 画+会員ロギ ン)	会員ログインが必要。backend/.envの MISSION_LOGIN_EMAIL/MISSION_LOGIN_PASSWORDでPlaywright から自動ログインしてスクレイピング

本番のDynamoDBテーブル・実際の各ブランド公式サイトへ直接アクセスするため、Lambdaとしてはデプロイせず、ローカル端末のAWS認証情報でpython backend/scripts/scrape_availability.py として手動実行、または端末側のタスクスケジューラ (cron/タスクスケジューラ) で定期実行する。

presignUploadHandler / updateStudioImageHandler / createPostHandler

目的：スタジオ写真・レビュー写真のアップロードと、レビュー投稿の作成。

処理フロー（写真アップロード）

1. フロントが POST /uploads/presign を呼び、S3への署名付きPOSTフォーム (uploadUrl + uploadUrl) を取得
2. フロントがそのフォームへ画像ファイルをmultipart/form-dataでPOST (Lambda/API Gatewayを経由)
3. アップロード完了後、公開URL (publicUrl) を
PUT /studios/{studioId}/image (スタジオ写真) または
POST /posts の imageUrl (レビュー写真) または
POST /chat の imageUrl (チャット添付) として保存

アップロード先S3バケット (UploadsBucket) はGetObjectのみ公開許可、書き込みは署名付きURL経由のみに限定している。generate_presigned_postのConditionsに ["content-length-range", 1, 8388608] (8MB) を指定し、S3側にアップロード上限を強制させている。

postChatHandler / getChatsHandler / getChatHandler

目的：写真添付にも対応したAIチャット機能。写真添付 (端末アルバムから選択 or その場で撮影、どちらも同じ<input type="file">で実現し、撮影用は capture="environment"属性を付与) をチャット入力の一部として統合している。

処理フロー（POST /chat）

1. UsageTableでレート制限チェック（1日30件、DAILY_CHAT_LIMIT）
2. editIndexが指定されていれば、該当インデックス以降のmessagesを切り捨ててから処理を続行
3. chatId が未指定なら新規チャットとしてuuid発行、指定されていればChatsTableから既存messages取得
4. ユーザーメッセージ（+ imageUrlがあればS3から画像バイトを取得）をmessagesに追記
5. _build_studios_context(lat, lng) がStudiosをscanし、実データグラウンディング用のコンテキストとして構築（旧来のRecommendations由来のスコア・reasonは含まない。スコアリング機能自体が廃止されたため、Studiosの実データのみを使う）
6. Claudeへ直近の会話履歴 + 今回のメッセージ（+画像）+ 実データコンテキストをマルチモーダル（システムプロンプトで「ダンス・ヨガの専門家アシスタント」という役割を明示。「データに無いスタジオは作り出さない」という指示も明記）
7. 応答をmessagesに追記し、ChatsTableへput_item（アイテム全体を上書き）
8. { chatId, reply, updatedAt } を同期的に返却

チャットはUX上ポーリングが合わないため**同期呼び出しのまま**とし、Lambda Timeoutを API Gatewayの29秒統合タイムアウトより短い25秒に設定することで対処している（テキスト+画像1枚のClaude呼び出しは通常数秒～十数秒で収まる想定）。

GET /chats は履歴パネルの一覧表示用に messages を含まない軽量なレスポンス（chatId/title/updatedAtのみ）を返す。GET /chats/{chatId} は履歴から会話を再開する際に全メッセージを返す。「新しい会話」開始時はDBに何も書き込まず、次にメッセージを送信した時点で初めて新規チャットとして保存される。

メッセージ編集: 自分の発言に表示される鉛筆アイコンから本文を訂正して再送信できる。フロント（ChatPage.tsx）が編集対象のインデックスを保持し、送信時にeditIndexとしてバックエンドへ渡す。postChatHandlerは該当インデックス以降の全メッセージ（訂正対象の発言+それ以降のAI応答）を切り捨ててから、訂正後の本文で通常の送信フローを再実行する（AIの回答も作り直される）。

postAnalyticsEventHandler / sendAnalyticsDigestBatch

目的：スタジオ詳細の表示・予約ボタンのクリックを記録し、日次で閲覧数・クリック数・コンバージョン率をメール通知する。認証不要（未ログインの閲覧行動も計測したいため）で、認証を経由しないリクエストのuserIdは常にDEFAULT_USER_IDになる。「予約完了」自体は外部の公式サイト側で行われるため計測できず、click_reserveは「予約ページへのクリック」までを示す点に注意。

このアプリの位置付けとしてAIスコアリングとは独立した機能であり、スコアリング機能の廃止による影響は受けていない。

コスト保護レート制限 (admin_trigger.py)

目的：現在地から探す（Google Places APIの呼び出しを伴い課金対象）について、AI相談（postChatHandlerのDAILY_CHAT_LIMIT）と同じ思想で1ユーザー1日あたりの上限を設けている。以前存在していたAI分析実行（ai-batchアクション）のレート制限は、対応する機能自体の削除にともない削除した。

関数	レート制限アクション名	1日の上限	設定方法
adminRunStudioDiscovery	studio-discovery	10回	template.yamlの RATE_LIMIT_ACTION/RATE_LIMIT_DAILY 環境変数

上限に達している場合はバッチ起動・Places API検索を一切行わずに429（{"status": "rate_limited", "message": "..."}）を返す。カウンタはUsageTable（userId + dateKey）にADD式でアトミック加算し、TTLで翌々日ごろ自動的に古いカウンタが削除される（postChatHandlerと全く同じ仕組み）。

getMyProfileHandler / putMyProfileHandler

目的：ユーザーが自分の表示名を設定・編集できるようにする。UsersTable（userId単一PK）に displayName・email・createdAt・updatedAt を保存する。GET /me は未登録でも404にせず displayName: null で200を返す。

投稿一覧（getPostsHandler）は、NoSQLはJOINできないので「アプリ側で userId→表示名の辞書を作って結合する」パターンでUsersTableをscanし、各投稿にauthorName（未設定ユーザーは"匿名"）を付与する。

5. 非機能要件

- API応答：1～2秒以内（チャットを除く）
- 集計バッチ：日次1回（sendAnalyticsDigestBatch）
- 空き状況の更新頻度：ローカルスクリプトの実行頻度に依存（クラウド側での保証なし）
- エラーハンドリング：外部API失敗時はフォールバック値で処理継続、例外を投げずバッチ全体を止めない

6. セキュリティ

6.1 認証範囲

閲覧は未ログインでも可能（スタジオ一覧・空き状況・地図・投稿一覧）。操作（お気に入り・投稿作成/削除・AI相談・イベント登録/削除/おすすめ検索・現在地から探す）はログイン必須という方針とした（誰でも見られるアプリという性質を保ちつつ、書き込み系・課金系だけを保護する狙い。釣行AIアプリの方針をそのまま踏襲）。

6.2 認証フロー

フロント（react-oidc-context, Authorization Code + PKCE）
 → Cognito Hosted UI（/oauth2/authorize）へリダイレクト
 → Hosted UI上でGoogleログイン（Cognito側のGoogle IdP連携経由）
 → Cognitoの/oauth2/idpresponseへコールバック → 認可コードをid_tokenに交換
 → 以後のAPIリクエストに Authorization: Bearer <id_token> を付与
 → API Gateway の CognitoAuthorizer（User Pool Authorizer）がid_tokenを検証
 → 検証成功時、Lambda側イベントに event.requestContext.authorizer.claims.sub が入る
 → handlers.py の _get_user_id(event) がこれを実ユーザーIDとして使用

CognitoはOIDC標準の end_session_endpoint をディスカバリドキュメントに含めないため、ログアウトはreact-oidc-contextの標準 signoutRedirect() ではなく、Hosted UI独自の /logout エンドポイントへ直接遷移する方式で実装している（frontend/src/auth/authConfig.ts の cognitoSignOut()。釣行AIアプリと同じ実装）。

6.3 所有者チェック

Favorites/Chats/Eventsテーブルは userId がpartition keyの一部のため、他人のuserIdではそもそも該当データにアクセスできない設計（追加の所有者チェック不要）。一方Postsテーブルは postId 単一PKで所有者情報がキーに含まれないため、

deletePostHandler/updatePostHandlerで更新・削除前に `get_item` して投稿の `userId` とリクエスト元の `userId` を比較し、一致しなければ403を返す 明示的なチェックを行っている。

7. テスト戦略

- バックエンド: pytest。純粋関数 (`haversine_km`・`detect_brand`・`_room_meets_size`・`_room_has_availability`等) の単体テストに加え、moto で DynamoDB/S3をモックした ハンドラーレベルのテスト (投稿・チャット・イベントの作成→削除ライフサイクル、写真アップロード、空き状況取得等)。約80件
- フロントエンド ユニットテスト: Vitest + React Testing Library (`utils/area.ts`・`utils/distance.ts`の純粋関数テスト)
- フロントエンド E2E・VRT: 導入していない (`package.json`に`@playwright/test`を含めていない)
- CI: `.github/workflows/test.yml` が `pull_request` → `main` をトリガーに実行される (`deploy.yml` の `push` → `main` トリガーとは分離。デプロイは行わずテストのみ。フロントエンドジョブは`npm run build`も実行し型エラーを検出する)

8. まとめ

Lambdaは「API処理」と「バッチ処理」に完全分離し、スケーラブルかつ低コストな構成とする。AIによるスコアリング・おすすめTOP3・全国自動発見の仕組みは廃止し、4ブランドの実店舗に絞った上で、公式サイトから収集した実際の空き状況とイベント (本番実寸・出演人数) を突き合わせるシンプルな絞り込み機能に置き換えている。AI (Claude) の役割はAI相談チャットの 応答生成のみに縮小した。